

# Team Midterm Report

<https://github.com/Dredman72/security-log-threat-timeline>

**Project:** Intelligent Security Log Summarization and Threat Timeline Generation

**Course:** CSC 482 Capstone Project 2

**Team:** Project Team 2

**Team Members:** Derrick Redman, Zion Moore, Oriah Molton-Bowman

**Reporting Period:** Weeks 1-4, June 1-June 28, 2026

## Project Overview

Our team project is a Flask-based security log analysis tool. The prototype allows a user to paste or upload security logs, sends the log content to the OpenAI API using the tested assignment model, and returns a structured security report. The report is designed to include an executive summary, risk level, attack type, affected assets, indicators of compromise, key findings, a chronological threat timeline with evidence, and recommended actions.

The goal is to help a security analyst or instructor understand raw security logs faster by reducing log noise, organizing events, and presenting the most important findings in a report-style format.

This midterm report combines the team's weekly journal progress from Week 1 through Week 4. It summarizes milestones achieved, subtasks completed, testing evidence, lessons learned, team member contributions, meeting attendance, and progress compared to the project plan.

## Current Status as of June 27, 2026

| Area                               | Status   | Notes                                                                                                                                                                           |
|------------------------------------|----------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Derrick Week 4 LLM work            | Complete | OpenAI prompt refinement, structured JSON output expectations, fallback handling, and upload handling review are complete.                                                      |
| Zion Week 4 backend work           | Complete | Parser output has been prepared in an AI-ready format so normalized events can support the OpenAI summarization workflow.                                                       |
| Oriah Week 3 frontend planning     | Complete | Frontend parsed-event planning and report/timeline layout planning are complete.                                                                                                |
| Oriah Week 4 report-section design | Complete | Oriah submitted report-section design evidence showing executive summary, key findings, recommendations, threat timeline, and raw event details for the frontend/report layout. |

## Team Meeting Attendance Log

The team used regular meetings to combine weekly work, review deliverables, coordinate GitHub updates, and keep the project aligned with the project plan. This midterm report combines the weekly journal progress from Weeks 1-4.

| Date          | Attendees            | Meeting Type / Topic                                                   | Time / Duration            |
|---------------|----------------------|------------------------------------------------------------------------|----------------------------|
| June 3, 2026  | Derrick, Zion, Oriah | Initial project discussion and team coordination                       | 4:33 PM - 5:35 PM, 1h 1m   |
| June 8, 2026  | Derrick, Zion, Oriah | Project planning and early prototype coordination                      | 3:53 PM - 4:48 PM, 54m     |
| June 10, 2026 | Derrick, Zion, Oriah | Project plan, roles, and prototype direction                           | 3:59 PM - 5:21 PM, 1h 22m  |
| June 17, 2026 | Derrick, Zion        | Backend/parser and validation coordination                             | 3:58 PM - 4:41 PM, 43m     |
| June 18, 2026 | Derrick, Zion, Oriah | Team review of project plan, parser fields, and frontend/report needs  | 3:29 PM - 6:45 PM, 3h 15m  |
| June 21, 2026 | Derrick, Zion        | Parser output review and Week 3 validation discussion                  | 10:31 AM - 11:40 AM, 1h 9m |
| June 22, 2026 | Derrick, Zion        | Week 4 parser-to-LLM coordination and report evidence planning         | 4:49 PM - 6:29 PM, 1h 43m  |
| June 24, 2026 | Derrick, Oriah       | Audio meeting about GitHub and frontend/report submission coordination | 26m                        |

## Milestones Achieved

| Done | Dates     | Milestone                                          | Brief Description                                                                                                                                                                                                         |
|------|-----------|----------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| [x]  | June 1-7  | Week 1: Project kickoff and requirements           | The team confirmed the project topic, team roles, initial scope, target log types, and the need for a runnable prototype.                                                                                                 |
| [x]  | June 8-14 | Week 2: Data model, sample logs, and output format | The team defined the shared backend, LLM, and frontend structure. Derrick created the LLM output format, Zion created the backend event schema and sample logs, and Oriah created the frontend report/timeline wireframe. |

| Done | Dates      | Milestone                                            | Brief Description                                                                                                                                                                                                          |
|------|------------|------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| [x]  | June 15-21 | Week 3: Log ingestion and parser validation          | Zion completed an initial parser prototype and normalized output. Derrick created the parser output validation checklist. The team reviewed the parser structure against the fields needed for the LLM and report display. |
| [x]  | June 22-28 | Week 4: LLM integration and structured summarization | Derrick refined the OpenAI prompt and fallback behavior. Zion prepared AI-ready parser input for the LLM workflow. Oriah completed the Week 4 frontend/report section design for the summary output.                       |

## Subtasks Completed

### Week 1 Subtasks

| Member  | Subtask                     | Brief Description                                                                                     |
|---------|-----------------------------|-------------------------------------------------------------------------------------------------------|
| Derrick | Initial prototype framework | Created the first Flask/OpenAI project structure with setup instructions and milestone TODO planning. |
| Derrick | Team coordination           | Confirmed the project goal and helped organize team responsibilities.                                 |
| Zion    | Backend planning            | Identified backend/log processing needs and the type of parser output required for the project.       |
| Oriah   | Frontend planning           | Identified the frontend report sections and timeline display needs.                                   |

### Week 2 Subtasks

| Member  | Subtask                          | Brief Description                                                                                                                                                                                |
|---------|----------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Derrick | LLM prompt and JSON output draft | Defined the report fields the OpenAI output should include: executive summary, risk level, attack type, affected assets, IOCs, key findings, timeline events, evidence, and recommended actions. |
| Zion    | Canonical event schema           | Defined parsed event fields such as timestamp, source, host, user, event type, severity, description, evidence, and raw event.                                                                   |
| Zion    | Sample parser data               | Created a sample CSV security log to support parser testing.                                                                                                                                     |
| Oriah   | Report/timeline wireframe        | Created a PowerPoint wireframe showing how the final report and timeline should be organized.                                                                                                    |

### Week 3 Subtasks

| Member  | Subtask                            | Brief Description                                                                                                                                                   |
|---------|------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Derrick | Parser output validation checklist | Created <code>PARSER_OUTPUT_VALIDATION_CHECKLIST.md</code> to verify that backend parser output supports the LLM prompt and frontend report.                        |
| Derrick | Parser review                      | Reviewed Zion's parser output and recommended improvements such as source IP, destination IP, standardized event types, and raw event preservation.                 |
| Zion    | Backend health check               | Verified the parser prototype health check returned status: ok.                                                                                                     |
| Zion    | CSV parser test                    | Tested the parser using sample CSV logs and confirmed it parsed 4 events and identified 3 High/Critical events.                                                     |
| Zion    | Normalized JSON output             | Generated parser output using the canonical event schema.                                                                                                           |
| Zion    | Starter report output              | Generated a starter HTML security report with an executive summary, validation warnings, and a threat timeline table.                                               |
| Oriah   | Frontend parsed-event planning     | Confirmed the needed parsed event fields, received the Flask/HTML framework direction, and completed Week 3 planning for a basic parsed-event frontend placeholder. |

### Week 4 Subtasks

| Member  | Subtask                             | Brief Description                                                                                                                                                             |
|---------|-------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Derrick | Refined OpenAI summarization prompt | Improved prompt instructions so the model returns concise, structured JSON report output.                                                                                     |
| Derrick | Improved fallback handling          | Added safer JSON parsing, report normalization, missing field defaults, and clear error messages for incomplete or invalid model output.                                      |
| Derrick | Upload handling review              | Clarified supported uploaded file types and added clear handling for raw .evtx files and oversized uploads.                                                                   |
| Zion    | Parser-to-LLM input preparation     | Prepared normalized parser output in an AI-ready format so parsed events can be passed into the OpenAI summarization workflow.                                                |
| Oriah   | Report section design               | Completed the Week 4 frontend/report section design by organizing the report around executive summary, key findings, recommendations, threat timeline, and raw event details. |

## Testing and Demo Evidence

The following tests were completed or prepared as evidence for the midterm report. Screenshots should be inserted into the submitted version of this report.

### Test 1: Flask Homepage Loads

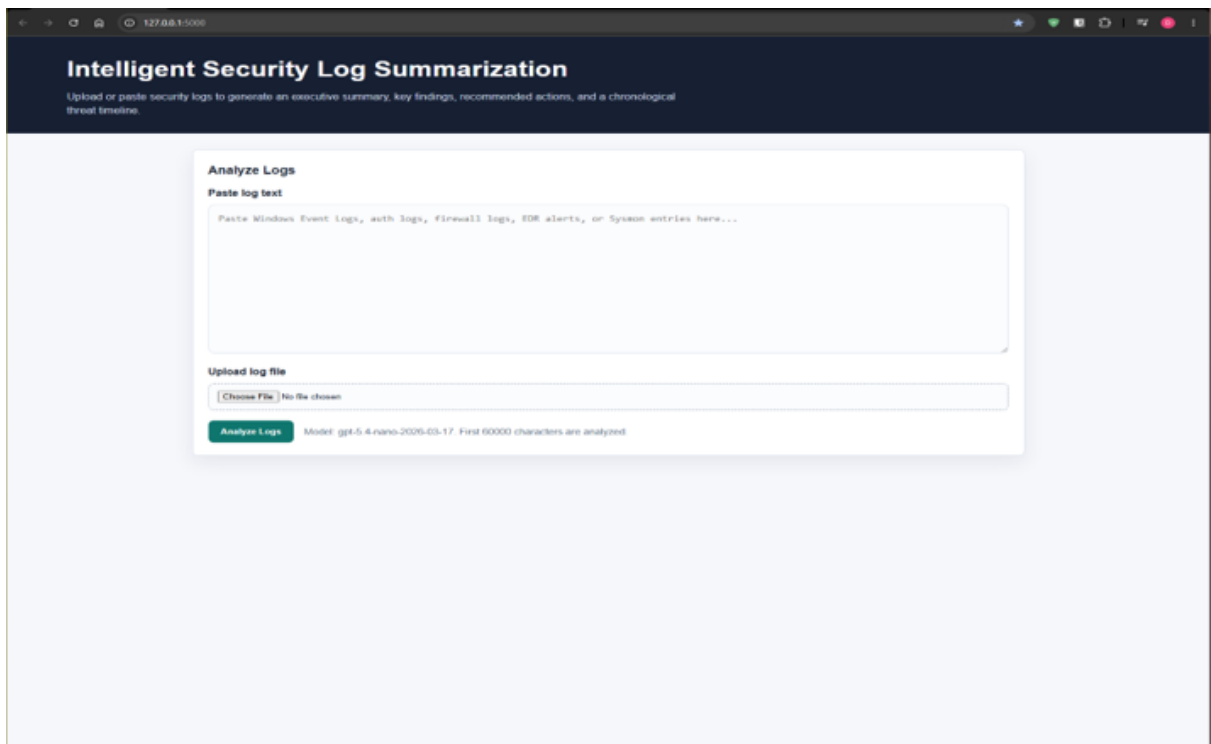
**Purpose:** Confirm the web application starts correctly and displays the log input form.

**Procedure:** The team ran the Flask application locally and opened `http://127.0.0.1:5000/`.

**Expected Result:** The page displays the application title, paste-log text box, file upload control, and Analyze Logs button.

**Result:** Passed. The Flask homepage loaded successfully.

**Screenshot Evidence:** Flask app homepage loaded successfully.



*Flask App Homepage*

## Test 2: OpenAI API Key and Usage

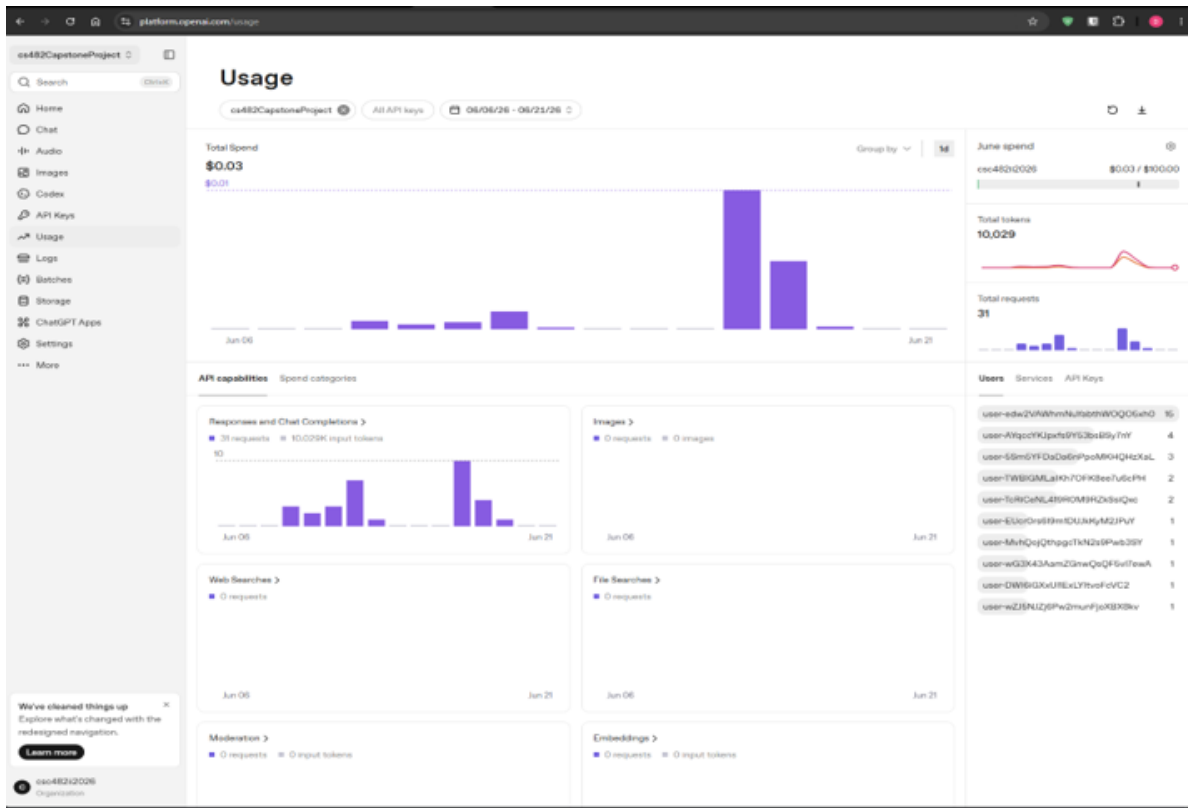
**Purpose:** Confirm the app can connect to OpenAI using the local .env API key without exposing the key.

**Procedure:** Derrick confirmed the API key exists locally, ran the application, submitted sample logs, and checked the OpenAI usage page.

**Expected Result:** OpenAI usage increases after analysis, and the application returns a report.

**Result:** Passed. OpenAI usage showed API activity, confirming the app sent requests successfully.

**Screenshot Evidence:** Insert screenshot of OpenAI Usage Dashboard.



OpenAI Usage Dashboard

### Test 3: Sample SSH and Firewall Log Analysis

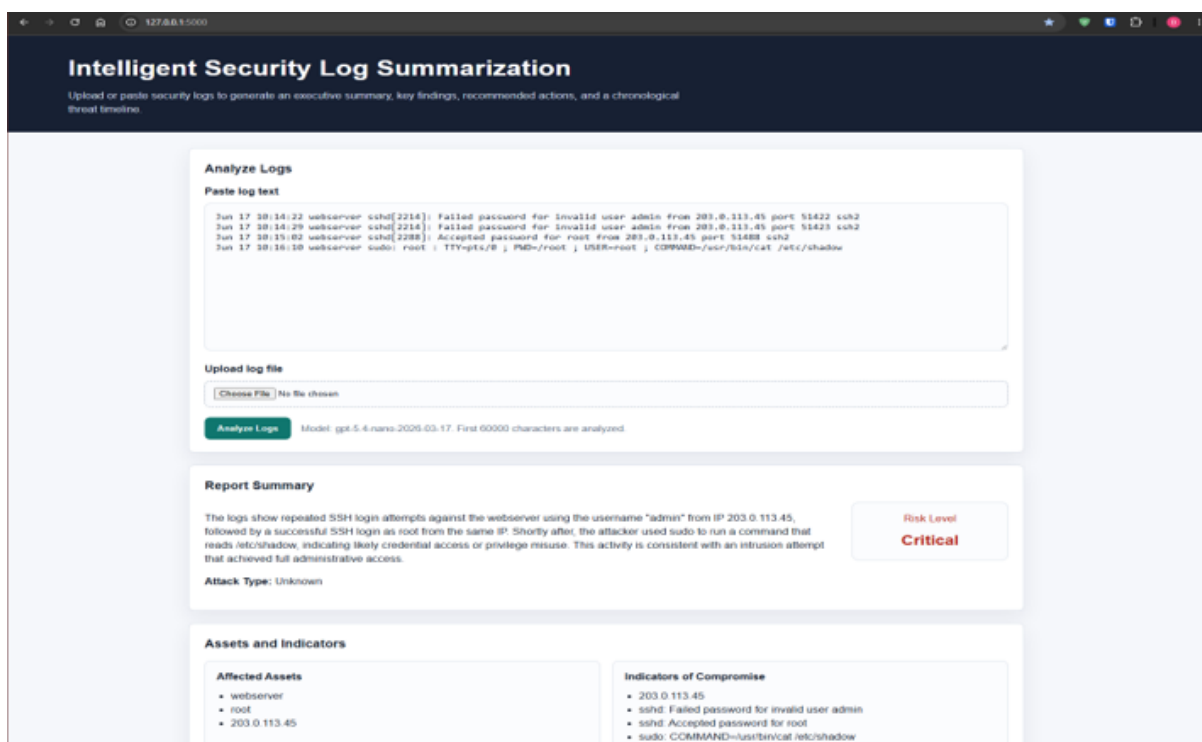
**Purpose:** Confirm the app can summarize suspicious security activity and generate report sections.

**Procedure:** Derrick pasted sample Linux SSH and firewall logs showing repeated failed SSH logins, a successful root login, a privileged command reading /etc/shadow, and a firewall block.

**Expected Result:** The app returns an executive summary, risk level, attack type, affected assets, indicators of compromise, key findings, timeline events with evidence, and recommended actions.

**Result:** Passed. The output was structured and demo-ready.

**Screenshot Evidence:** Insert screenshot showing sample logs pasted into the app and report output.



Sample logs pasted into app and report output

## Test 4: Threat Timeline Evidence

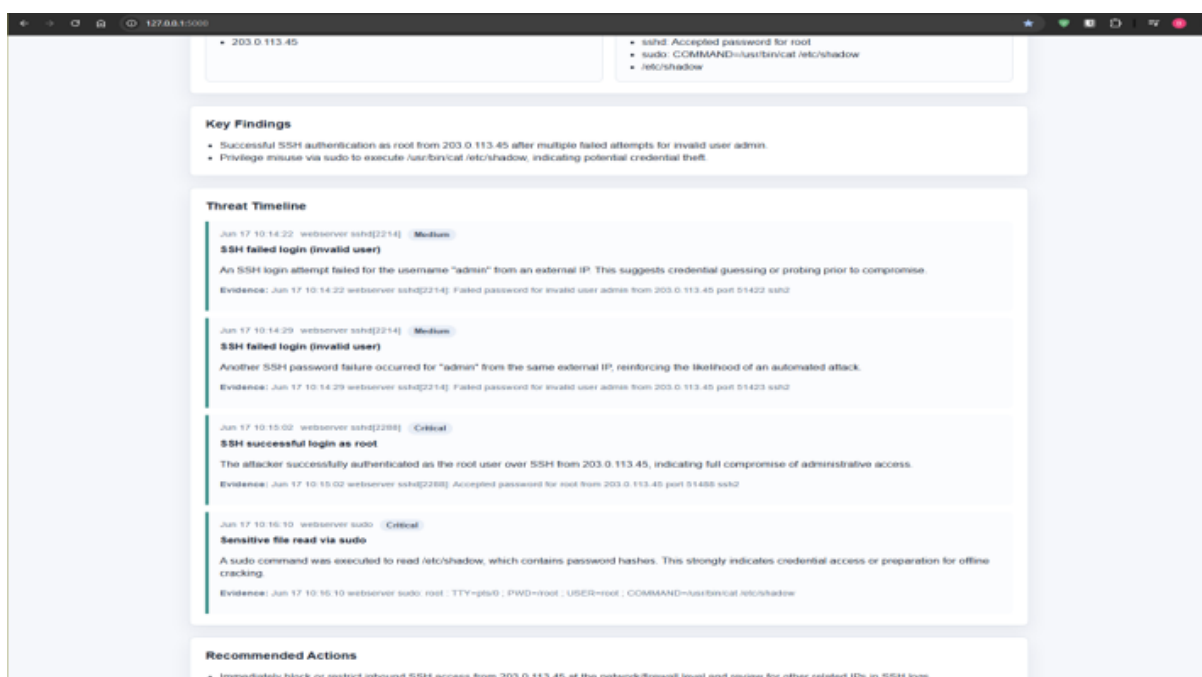
**Purpose:** Confirm timeline events include timestamp, source, severity, details, and supporting evidence.

**Procedure:** The team reviewed the generated timeline section after submitting sample logs.

**Expected Result:** Timeline events are chronological and include evidence from the original log lines.

**Result:** Passed. Timeline entries included timestamps, source fields, severity badges, details, and evidence.

**Screenshot Evidence:** Insert screenshot of threat timeline section.



Threat Timeline section



## Test 5: Parser Output Validation

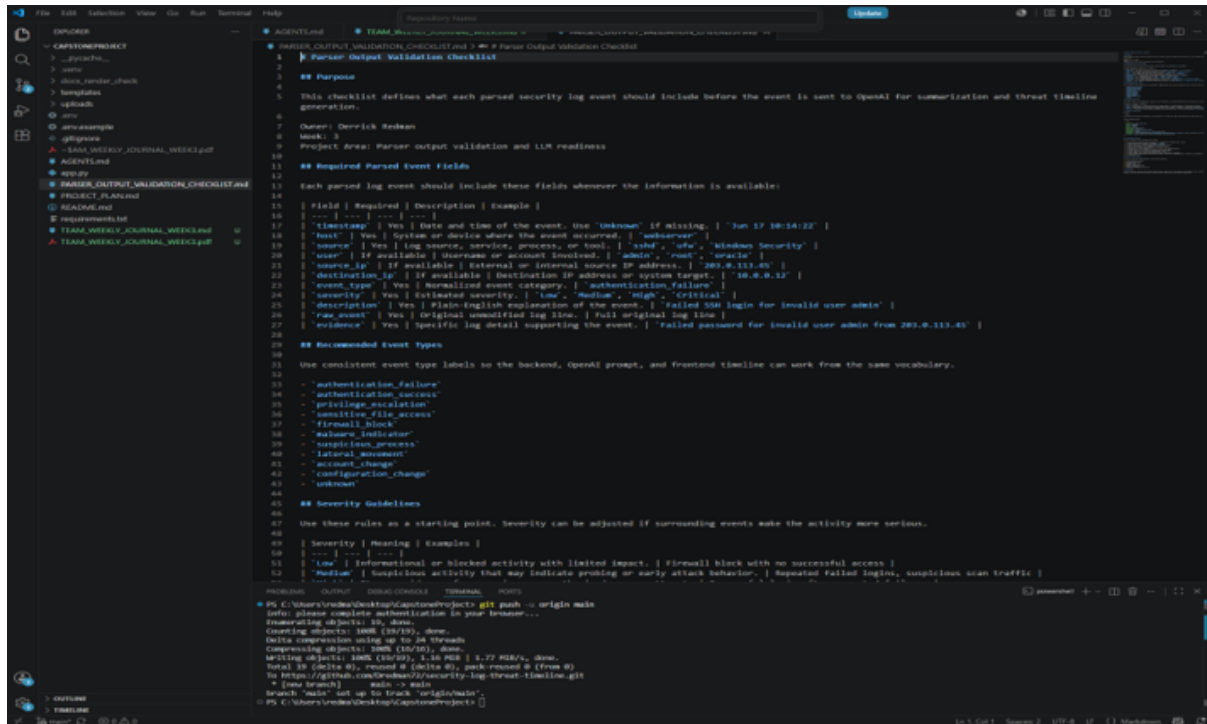
**Purpose:** Confirm Zion's parser output matches the fields needed by the LLM and frontend.

**Procedure:** Derrick reviewed Zion's normalized parser output against `PARSER_OUTPUT_VALIDATION_CHECKLIST.md`.

**Expected Result:** Parser output includes required fields such as timestamp, source, host, user, event type, severity, description, evidence, and raw event.

**Result:** Passed with improvement notes. Derrick recommended adding `source_ip` and `destination_ip` when available, standardizing event type names, and preserving the original raw event.

**Screenshot Evidence:** Insert screenshot of parser output and checklist review.



```
1 # PARSER OUTPUT VALIDATION CHECKLIST and a # Parser Output Validation Checklist
2 # Parser Output Validation Checklist
3
4 ## Purpose
5 This checklist defines what each parsed security log event should include before the event is sent to OpenAI for summarization and threat timeline generation.
6
7 Author: Derrick Redman
8 Week: 5
9 Project Area: Parser output validation and LLM readiness
10
11 ## Required Parsed Event Fields
12 Each parsed log event should include these fields whenever the information is available:
13
14 | Field | Required | Description | Example |
15 | --- | --- | --- | --- |
16 | timestamp | Yes | Date and time of the event. Use 'unknown' if missing. | 'Jan 17 10:14:22' |
17 | host | Yes | System or device where the event occurred. | 'adserver' |
18 | source | Yes | Log source, service, process, or tool. | 'sshd', 'smb', 'Windows Security' |
19 | user | If available | Username or account identifier. | 'admin', 'root', 'guest' |
20 | source_ip | If available | External or internal source IP address. | '203.0.113.45' |
21 | destination_ip | If available | Destination IP address or system target. | '10.0.0.12' |
22 | event_type | Yes | Normalized event category. | 'authentication_failure' |
23 | severity | Yes | Estimated severity. | 'low', 'medium', 'high', 'critical' |
24 | description | Yes | Plain-English explanation of the event. | 'Failed SSH login for invalid user admin' |
25 | raw_event | Yes | Original unmodified log line. | Full original log line |
26 | evidence | Yes | Specific log detail supporting the event. | 'Failed password for invalid user admin from 203.0.113.45' |
27
28 ## Recommended Event Types
29 Use consistent event type labels so the backend, OpenAI prompt, and frontend timeline can work from the same vocabulary.
30
31 - 'authentication_failure'
32 - 'authentication_success'
33 - 'privilege_escalation'
34 - 'sensitive_file_access'
35 - 'firewall_block'
36 - 'malware_indicator'
37 - 'suspicious_process'
38 - 'lateral_movement'
39 - 'account_change'
40 - 'configuration_change'
41 - 'unknown'
42
43 ## Severity Guidelines
44 Use these rules as a starting point. Severity can be adjusted if surrounding events make the activity more serious.
45
46 | Severity | Meaning | Examples |
47 | --- | --- | --- |
48 | 'low' | Informational or blocked activity with limited impact. | Firewall block with no successful access |
49 | 'medium' | Suspicious activity that may indicate probing or early attack behavior. | Repeated failed logins, suspicious scan traffic |
50 | 'high' | Active attack or confirmed compromise. | Successful privilege escalation, malware execution |
51 | 'critical' | Severe security incident or data breach. | Successful ransomware deployment, data exfiltration |
52
53 ## Notes
54 - 'auth' - Authentication events
55 - 'conn' - Connection events
56 - 'exec' - Execution events
57 - 'file' - File system events
58 - 'net' - Network events
59 - 'proc' - Process events
60 - 'sys' - System events
61 - 'tool' - Tool or application events
62 - 'user' - User-related events
63 - 'vuln' - Vulnerability-related events
64 - 'warn' - Warning or informational events
65
66 ## Footer
67 This document is part of the Zion Project. For more information, see the project documentation at https://zionproject.org/docs.
68
69 ## License
70 This document is licensed under the Creative Commons Attribution-NonCommercial-ShareAlike 4.0 International License.
71
72 ## Version
73 1.0.0
74
75 ## History
76 2024-01-17: Initial draft by Derrick Redman
77 2024-01-18: Added 'evidence' field and 'firewall_block' event type
78 2024-01-19: Final review and formatting
79
80 ## Contact
81 Derrick Redman: derrick@zionproject.org
82
83 ## Acknowledgments
84 Thanks to the Zion Project community for their support and feedback.
```

```
PS C:\Users\redman\Desktop\zionproject> git push --origin main
Enumerating objects: 1008 (10/10), done.
Delta compression using up to 26 threads
Compressing objects: 100% (10/10), done.
Writing objects: 100% (10/10), 1.5 KiB | 1.77 MiB/s, done.
Total 10 (delta 0), reused 0 (delta 0), pack reused 0 (from 0)
To https://github.com/dredman02/zionproject/security-log-threat-timeline.git
 * [new branch]      main -> main
branch 'main' set up to track 'origin/main'.
PS C:\Users\redman\Desktop\zionproject>
```

Parser output Validation Checklist review

```
1 {
2   "generated_at": "2026-06-17T21:40:35.893921+00:00",
3   "events": [
4     {
5       "timestamp": "2026-06-09T13:00:00Z",
6       "source": "demo-sysmon",
7       "host": "WIN-SERVER-01",
8       "user": "alice",
9       "event_type": "process_start",
10      "severity": "High",
11      "description": "PowerShell launched with unusual encoded command",
12      "evidence": "Commandline contains encoded command",
13      "raw_event": "{\"timestamp\": \"2026-06-09T13:00:00Z\", \"source\": \"demo-sysmon\", \"host\": \"WIN-SERVER-01\", \"user\": \"alice\", \"event_type\": \"process_start\", \"severity\": \"High\", \"description\": \"PowerShell launched with unusual encoded command\", \"evidence\": \"Commandline contains encoded command\", \"raw_event\": \"\"}",
14    },
15    {
16      "timestamp": "2026-06-09T13:05:00Z",
17      "source": "demo-firewall",
18      "host": "WIN-SERVER-01",
19      "user": "unknown",
20      "event_type": "firewall_block",
21      "severity": "High",
22      "description": "Outbound connection blocked to suspicious IP",
23      "evidence": "Destination IP 203.0.113.10 blocked",
24      "raw_event": "{\"timestamp\": \"2026-06-09T13:05:00Z\", \"source\": \"demo-firewall\", \"host\": \"WIN-SERVER-01\", \"user\": \"unknown\", \"event_type\": \"firewall_block\", \"severity\": \"High\", \"description\": \"Outbound connection blocked to suspicious IP\", \"evidence\": \"Destination IP 203.0.113.10 blocked\", \"raw_event\": \"\"}",
25    },
26    {
27      "timestamp": "2026-06-09T13:11:00Z",
28      "source": "demo-windows-event",
29      "host": "WIN-SERVER-01",
30      "user": "bob",
31      "event_type": "login_failure",
32      "severity": "Medium",
33      "description": "Multiple failed login attempts detected",
34      "evidence": "Three failed attempts from same account",
35      "raw_event": "{\"timestamp\": \"2026-06-09T13:11:00Z\", \"source\": \"demo-windows-event\", \"host\": \"WIN-SERVER-01\", \"user\": \"bob\", \"event_type\": \"login_failure\", \"severity\": \"Medium\", \"description\": \"Multiple failed login attempts detected\", \"evidence\": \"Three failed attempts from same account\", \"raw_event\": \"\"}",
36    },
37    {
38      "timestamp": "2026-06-09T13:20:00Z",
39      "source": "demo-edr",
40      "host": "WIN-SERVER-01",
41      "user": "alice",
42      "event_type": "malware_alert",
43      "severity": "Critical",
44      "description": "EDR reported possible ransomware behavior",
45      "evidence": "Process attempted rapid file encryption",
46      "raw_event": "{\"timestamp\": \"2026-06-09T13:20:00Z\", \"source\": \"demo-edr\", \"host\": \"WIN-SERVER-01\", \"user\": \"alice\", \"event_type\": \"malware_alert\", \"severity\": \"Critical\", \"description\": \"EDR reported possible ransomware behavior\", \"evidence\": \"Process attempted rapid file encryption\", \"raw_event\": \"\"}",
47    },
48  ]
49 }
```

Zions parsed output file

## Test 6: Fallback Behavior Review

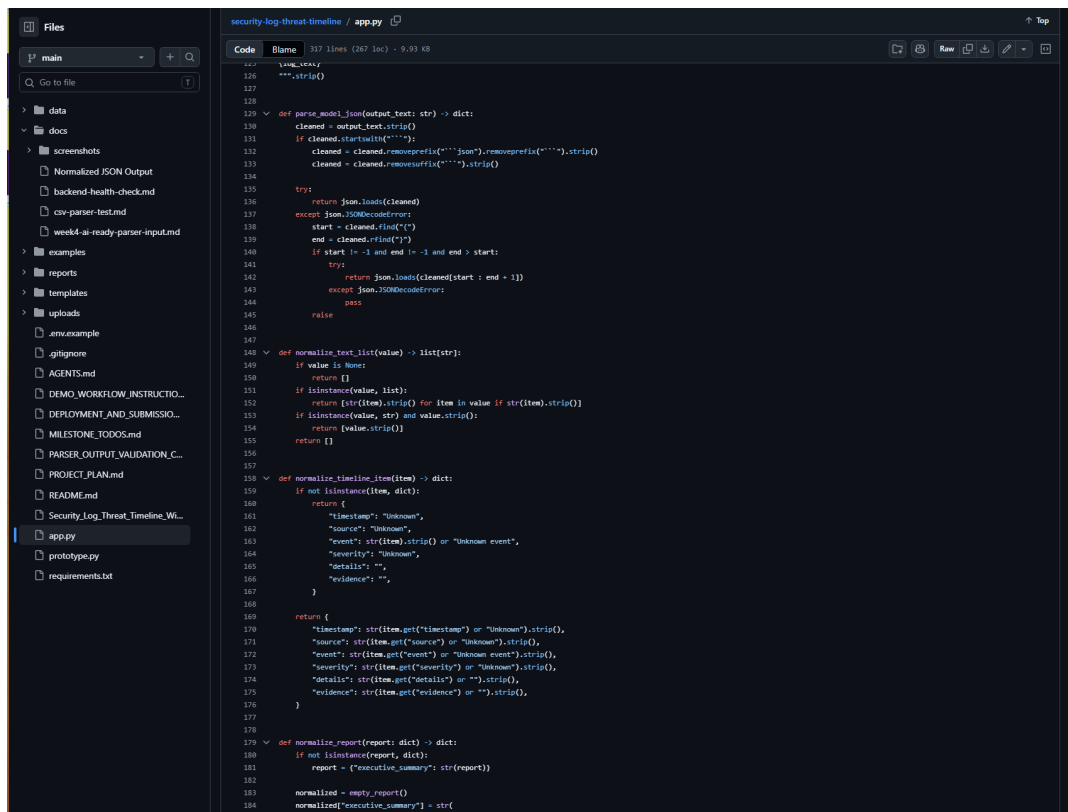
**Purpose:** Confirm the app handles incomplete or imperfect OpenAI output gracefully.

**Procedure:** Derrick updated the app so model output is normalized before display and missing fields are filled with safe defaults.

**Expected Result:** The app does not crash when the model output is missing fields, incomplete, or wrapped in formatting.

**Result:** Passed. The app includes JSON cleanup, report normalization, missing-field defaults, and an invalid-JSON fallback message. This prevents the app from silently failing when OpenAI output is incomplete or imperfect

**Screenshot Evidence:** Insert screenshot of successful report output or terminal check if used.



### Fallback Imperfect Output Code

- It asks OpenAI for JSON directly:
 

```
python
response_format={"type": "json_object"}
```
- It cleans model output if the model wraps JSON in a code block:
 

```
python
if cleaned.startswith("```"):
    cleaned = cleaned.removeprefix("```").strip()
    cleaned = cleaned.removeprefix("```").strip()
```
- It tries to recover JSON even if extra text appears before/after it:
 

```
python
start = cleaned.find("(")
end = cleaned.rfind(")")
```
- It normalizes missing fields instead of crashing:
 

```
python
normalized["risk_level"] = str(report.get("risk_level") or "Unknown").strip()
normalized["attack_type"] = str(report.get("attack_type") or "Unknown").strip()
```
- It fills missing timeline fields safely:
 

```
python
"timestamp": str(item.get("timestamp") or "Unknown").strip()
"event": str(item.get("event") or "Unknown event").strip()
```
- It adds a warning if required report fields are missing:
 

```
python
Review output: missing expected field(s)
```
- If OpenAI returns invalid JSON, it falls back to a readable report instead of crashing:
 

```
python
"Review the raw model output because it was not valid JSON."
```

### Fallback Codes

## Test 7: AI-Ready Parser Input Review

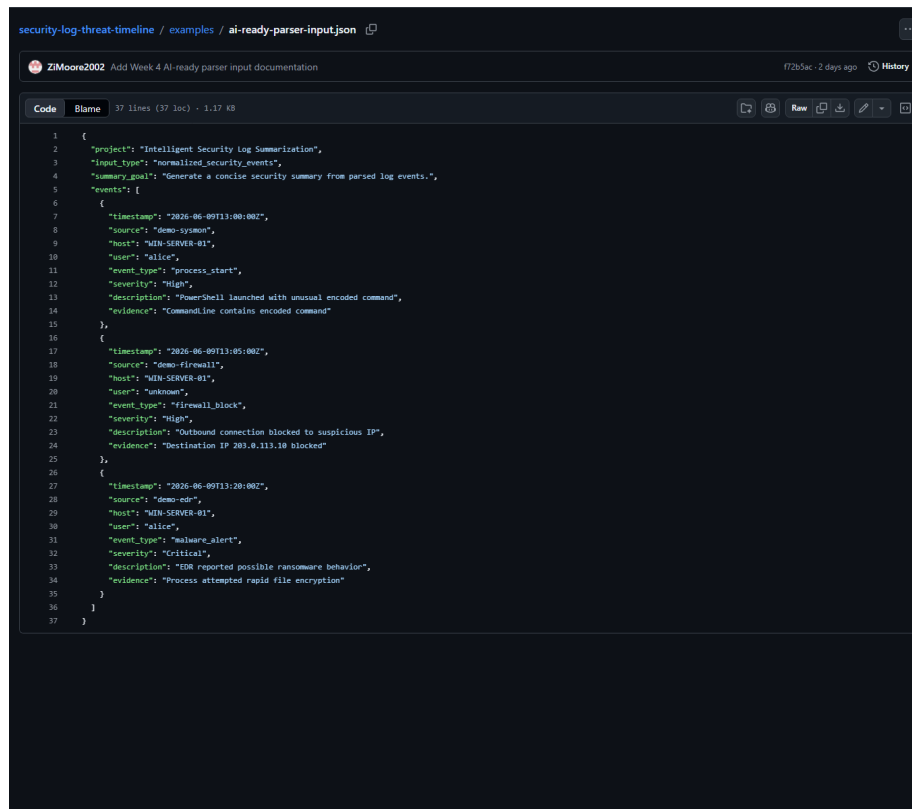
**Purpose:** Confirm Zion's normalized parser output can support Derrick's OpenAI summarization workflow.

**Procedure:** The team reviewed the AI-ready parser input structure against the expected LLM/report fields.

**Expected Result:** Parser output should include enough normalized event detail to support summary, risk level, key findings, timeline events, evidence, and recommended actions.

**Result:** Passed. Zion's Week 4 parser-to-LLM input work is complete for the current prototype stage.

**Screenshot Evidence:** Zion's AI-ready parser input file shows normalized security events prepared for Derrick's OpenAI summarization workflow. The file includes the summary goal and event fields needed for LLM reporting, including timestamp, source, host, user, event\_type, severity, description, and evidence.



```
1 {
2   "project": "Intelligent Security Log Summarization",
3   "input_type": "normalized_security_events",
4   "summary_goal": "Generate a concise security summary from parsed log events.",
5   "events": [
6     {
7       "timestamp": "2026-06-09T13:00:00Z",
8       "source": "demo-system",
9       "host": "WIN-SERVER-01",
10      "user": "alice",
11      "event_type": "process_start",
12      "severity": "High",
13      "description": "PowerShell launched with unusual encoded command",
14      "evidence": "CommandLine contains encoded commands"
15    },
16    {
17      "timestamp": "2026-06-09T13:05:00Z",
18      "source": "demo-firewall",
19      "host": "WIN-SERVER-01",
20      "user": "unknown",
21      "event_type": "firewall_block",
22      "severity": "High",
23      "description": "Outbound connection blocked to suspicious IP",
24      "evidence": "Destination IP 203.0.113.10 blocked"
25    },
26    {
27      "timestamp": "2026-06-09T13:20:00Z",
28      "source": "demo-edr",
29      "host": "WIN-SERVER-01",
30      "user": "alice",
31      "event_type": "malware_alert",
32      "severity": "Critical",
33      "description": "EDR reported possible ransomware behavior",
34      "evidence": "Process attempted rapid file encryption"
35    }
36  ]
37 }
```

Test 7: AI-Ready Parser Input Review

## Test 8: Week 4 Frontend Report Section Design

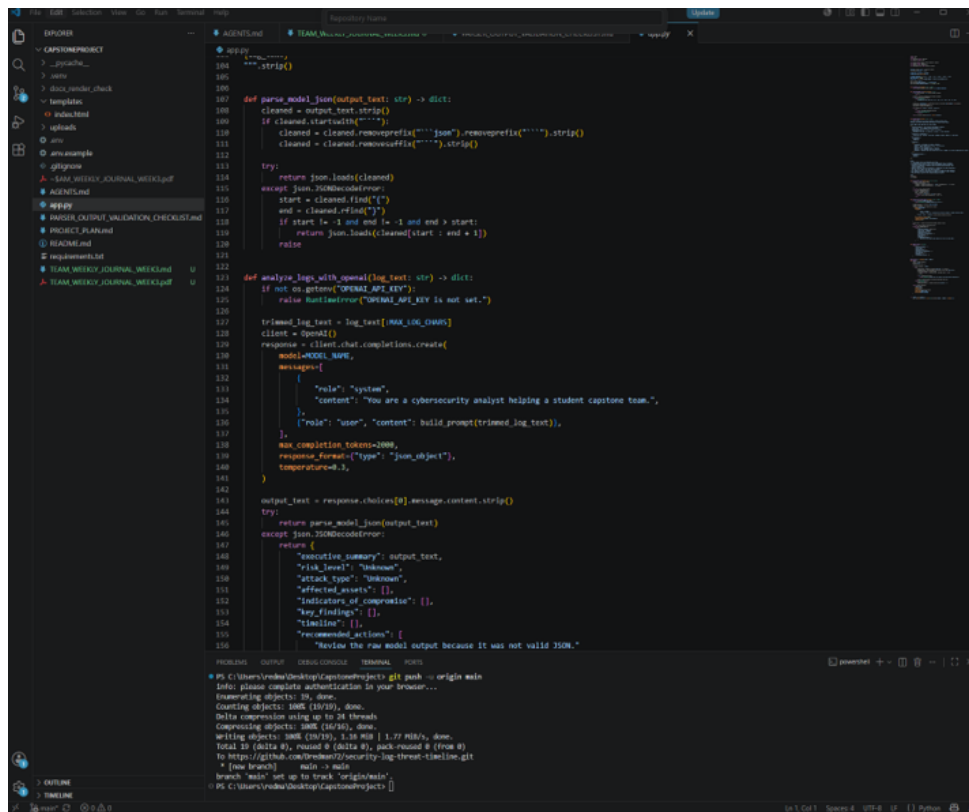
**Purpose:** Confirm Oriah's Week 4 frontend/report section design is ready for the shared Flask/HTML prototype.

**Procedure:** The team reviewed Oriah's report-section design evidence and confirmed that the layout includes the major report sections needed by the project.

**Expected Result:** The report design should clearly support executive summary, key findings, recommendations, threat timeline, and raw event details.

**Result:** Passed. Oriah's Week 4 frontend/report section design is complete for the current prototype stage.

**Screenshot Evidence:** Oriah's structured report output planning shows how the frontend/report layout should organize summary, findings, timeline, and recommendations.



```
def parse_model_json(output_text: str) -> dict:
    cleaned = output_text.strip()
    if cleaned.startswith("```"):
        cleaned = cleaned.removeprefix("```").removeprefix("json").removeprefix("```").strip()
        cleaned = cleaned.removeprefix("```").strip()
    try:
        return json.loads(cleaned)
    except json.JSONDecodeError:
        start = cleaned.find("{")
        end = cleaned.rfind("}")
        if start != -1 and end != -1 and end > start:
            return json.loads(cleaned[start : end + 1])
        raise

def analyze_logs_with_openai(log_text: str) -> dict:
    if not os.getenv("OPENAI_API_KEY"):
        raise RuntimeError("OPENAI_API_KEY is not set.")

    trimmed_log_text = log_text[LOG_IGNORE_CHUNKS:]
    client = OpenAI()
    response = client.chat.completions.create(
        model=MODEL_NAME,
        messages=[
            {
                "role": "system",
                "content": "You are a cybersecurity analyst helping a student capture team.",
            },
            {
                "role": "user",
                "content": build_prompt(trimmed_log_text),
            },
        ],
        max_completion_tokens=2000,
        response_format={"type": "json_object"},
        temperature=0.3,
    )

    output_text = response.choices[0].message.content.strip()
    try:
        return parse_model_json(output_text)
    except json.JSONDecodeError:
        return {
            "executing_summary": output_text,
            "risk_level": "unknown",
            "attack_type": "unknown",
            "affected_hosts": [],
            "indicators_of_compromise": [],
            "key_findings": [],
            "timeline": [],
            "recommended_actions": [
                "Review the raw model output because it was not valid JSON."
            ]
        }
```

Structured Report Output Planning

## Lessons Learned

- The team learned that the backend parser, LLM prompt, and frontend report must share the same data structure.
- Derrick learned that structured prompt design is important because the frontend depends on consistent report fields.
- Zion's parser work showed that normalized event fields are necessary before the system can reliably summarize logs.
- Oriah's wireframe showed that frontend planning helps define what backend and LLM fields are needed.
- The team learned that GitHub collaboration requires clean file organization, clear commit messages, and avoiding private files such as .env.
- The team also learned that local testing and VM deployment are different. 127.0.0.1 works only on the computer running the app, while VM deployment requires the app to listen on the VM network and use port 80 or 443.

## Team Member Contributions

| Team Member         | Role                            | Contributions So Far                                                                                                                                                                                                                                                                                                                |
|---------------------|---------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Derrick Redman      | Team Leader + AI/LLM Specialist | Created the Flask/OpenAI prototype, confirmed the tested OpenAI model, refined the LLM prompt, improved fallback handling, created the parser validation checklist, updated README/project documentation, created GitHub repository, coordinated team direction, and reviewed team deliverables.                                    |
| Zion Moore          | Backend & Log Processing Lead   | Created the canonical event schema, created sample CSV security logs, built the first parser prototype, ran backend health checks, parsed sample events, identified High/Critical events, generated normalized JSON output, produced a starter HTML security report, and prepared AI-ready parser input for Week 4 LLM integration. |
| Oriah Molton-Bowman | Frontend & Visualization Lead   | Created the Week 2 PowerPoint report/timeline wireframe, planned frontend sections, identified frontend data fields, planned timeline display behavior, requested parsed sample data, completed Week 3 parsed-event/frontend planning, and completed the Week 4 report-section design for summary output.                           |

## Progress Compared to Plan

The team has progressed according to the project plan, timeline, and schedule.

Weeks 1, 2, 3, and 4 are complete for the current midterm checkpoint. Derrick's Week 4 LLM prompt and fallback handling tasks are complete. Zion's Week 4 parser-to-LLM input preparation is complete. Oriah's Week 4 report-section design task is complete.

## Plan Adjustment

No major plan adjustment is required. The team will continue using Flask and HTML as the shared framework. Since the Week 4 work is complete, the next coordination step is to move into Week 5 timeline generation work: Derrick will define timeline wording and evidence rules, Zion will support chronological sorting/grouping, and Oriah will begin the interactive timeline prototype.

The Week 5 focus remains threat timeline generation logic and interactive timeline prototype work.

## Team Sign-Off

Each team member must sign before submission.

| Team Member         | Role                            | Signature | Date |
|---------------------|---------------------------------|-----------|------|
| Derrick Redman      | Team Leader + AI/LLM Specialist |           |      |
| Zion Moore          | Backend & Log Processing Lead   |           |      |
| Oriah Molton-Bowman | Frontend & Visualization Lead   |           |      |